

Chapter II

Goals

This project's objectives are similar to all this first year's objectives: rigor, use of C, basic algorithms, information research, etc.

As a graphic design project, **cub3D** will enable you to improve your skills in these areas: windows, colors, events, fill shapes, etc.

In conclusion, **cub3D** is a remarkable playground to explore the playful practical applications of mathematics without having to understand the specifics.

With the help of numerous documents available on the internet, you will use mathematics as a tool to create elegant and efficient algorithms.



If this suits you, you can test the original game before starting this project:

<http://users.atw.hu/wolf3d/>

Chapter III

Common Instructions

- Your project must be written in C.
- Your project must be written in accordance with the Norm. If you have bonus files/functions, they are included in the norm check, and you will receive a 0 if there is a norm error.
- Your functions should not quit unexpectedly (segmentation fault, bus error, double free, etc.) except for undefined behavior. If this occurs, your project will be considered non-functional and will receive a 0 during the evaluation.
- All heap-allocated memory must be properly freed when necessary. Memory leaks will not be tolerated.
- If the subject requires it, you must submit a **Makefile** that compiles your source files to the required output with the flags **-Wall**, **-Wextra**, and **-Werror**, using **cc**. Additionally, your **Makefile** must not perform unnecessary relinking.
- Your **Makefile** must contain at least the rules **\$(NAME)**, **all**, **clean**, **fclean** and **re**.
- To submit bonuses for your project, you must include a **bonus** rule in your **Makefile**, which will add all the various headers, libraries, or functions that are not allowed in the main part of the project. Bonuses must be placed in **_bonus.{c/h}** files, unless the subject specifies otherwise. The evaluation of mandatory and bonus parts is conducted separately.
- If your project allows you to use your **libft**, you must copy its sources and its associated **Makefile** into a **libft** folder. Your project's **Makefile** must compile the library by using its **Makefile**, then compile the project.
- We encourage you to create test programs for your project, even though this work **does not need to be submitted and will not be graded**. It will give you an opportunity to easily test your work and your peers' work. You will find these tests especially useful during your defence. Indeed, during defence, you are free to use your tests and/or the tests of the peer you are evaluating.
- Submit your work to the assigned Git repository. Only the work in the Git repository will be graded. If Deepthought is assigned to grade your work, it will occur

after your peer-evaluations. If an error happens in any section of your work during Deepthought's grading, the evaluation will stop.

Chapter IV

AI Instructions

● Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

● Main message

- 👉 Use AI to reduce repetitive or tedious tasks.
- 👉 Develop prompting skills — both coding and non-coding — that will benefit your future career.
- 👉 Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- 👉 Continue building both technical and power skills by working with your peers.
- 👉 Only use AI-generated content that you fully understand and can take responsibility for.

● Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting — this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.
- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review — don't rely solely on your own validation.

● Phase outcomes:

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

● Comments and examples:

- You'll regularly encounter situations — exams, evaluations, and more — where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

✓ Good practice:

I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

✗ Bad practice:

I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility — and I fail my project.

✓ Good practice:

I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together — better, cleaner, and fully understood.

✗ Bad practice:

I let Copilot generate my code for a key part of my project. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my project.

Chapter V

Mandatory part - cub3D

Program Name	cub3D
Files to Submit	All your files
Makefile	all, clean, fclean, re, bonus
Arguments	a map in format *.cub
External Function	• open, close, read, write, printf, malloc, free, perror, strerror, exit, gettimeofday. • All functions of the math library (-lm man man 3 math). • gettimeofday() • All functions of the MinilibX library.
Libft authorized	Yes
Description	You must create a "realistic" 3D graphical representation of the inside of a maze from a first-person perspective. You have to create this representation using the ray-casting principles mentioned earlier.

The constraints are as follows:

- You must use the miniLibX. Either the version that is available on the operating system, or from its sources. If you choose to work with the sources, you will need to apply the same rules for your libft as those written above in Common Instructions part.
- The management of your window must remain smooth: changing to another window, minimizing, etc.
- Display different wall textures (the choice is yours) that vary depending on which side the wall is facing (North, South, East, West).

- Your program must be able to set the floor and ceiling colors to two different ones.
- The program displays the image in a window and respects the following rules:
 - The left and right arrow keys of the keyboard must allow you to look left and right in the maze.
 - The W, A, S, and D keys must allow you to move the point of view through the maze.
 - Pressing **ESC** must close the window and quit the program cleanly.
 - Clicking on the red cross on the window's frame must close the window and quit the program cleanly.
 - The use of `images` of the `minilibX` library is strongly recommended.
- Your program must take as a first argument a scene description file with the `.cub` extension.
 - The map must be composed of only 6 possible characters: **0** for an empty space, **1** for a wall, and **N,S,E** or **W** for the player's start position and spawning orientation.

This is a simple valid map:

```
111111
100101
101001
1100N1
111111
```

- The map must be closed/surrounded by walls, if not the program must return an error.
- Except for the map content, each type of element can be separated by one or more empty lines.
- Except for the map content which always has to be the last, each type of element can be set in any order in the file.
- Except for the map, each type of information from an element can be separated by one or more spaces.
- The map must be parsed as it looks in the file. Spaces are a valid part of the map and are up to you to handle. You must be able to parse any kind of map, as long as it respects the rules of the map.

- Except for the map, each element must begin with its type identifier (composed by one or two characters), followed by its specific information in a strict order:

- * North texture:

```
NO ./path_to_the_north_texture
```

- identifier: **NO**
- path to the north texture

- * South texture:

```
SO ./path_to_the_south_texture
```

- identifier: **SO**
- path to the south texture

- * West texture:

```
WE ./path_to_the_west_texture
```

- identifier: **WE**
- path to the west texture

- * East texture:

```
EA ./path_to_the_east_texture
```

- identifier: **EA**
- path to the east texture

- * Floor color:

```
F 220,100,0
```

- identifier: **F**
- R,G,B colors in range [0,255]: **0, 255, 255**

* Ceiling color:

```
C 225,30,0
```

- identifier: **C**
- R,G,B colors in range [0,255]: **0, 255, 255**

○ Example of the mandatory part with a minimalist **.cub** scene:

```
NO ./path_to_the_north_texture
SO ./path_to_the_south_texture
WE ./path_to_the_west_texture
EA ./path_to_the_east_texture

F 220,100,0
C 225,30,0

11111111111111111111111111111111
10000000001100000000000001
10110000011100000000000001
10010000000000000000000001
11111111101100000111000000000001
100000000001100000111011111111111
11110111111111011100000010001
11110111111111011101010010001
11000000110101011100000010001
100000000000000001100000010001
100000000000000001101010010001
1100000111010101111011110N0111
11110111 1110101 101111010001
11111111 1111111 111111111111
```

○ If any misconfiguration of any kind is encountered in the file, the program must exit properly and return "Error\n" followed by an explicit error message of your choice.